



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

(FIAP)

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

GRUPO SÃO PAULO E INTERIOR

JONAS TADEU V. FERNANDES

LEVI PASSOS SILVEIRA MARQUES

RAPHAEL DA SILVA

RAPHAEL DINELLI NETO

YAN PIMENTAL COTTA

CardiolA:

Transmissão para nuvem e visualização (Fog/Cloud Computing)

SÃO PAULO

2026



**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
(FIAP)**

**CURSO SUPERIOR DE TECNOLOGIA EM INTELIGÊNCIA
ARTIFICIAL**

**GRUPO SÃO PAULO E INTERIOR
JONAS TADEU V. FERNANDES
LEVI PASSOS SILVEIRA MARQUES
RAPHAEL DA SILVA
RAPHAEL DINELLI NETO
YAN PIMENTAL COTTA**

CardiolA:

Transmissão para nuvem e visualização (Fog/Cloud Computing)

Trabalho acadêmico apresentado ao Curso Superior de Tecnologia em Inteligência Artificial da Faculdade de Informática e Administração Paulista (FIAP), como requisito para obtenção de nota na disciplina Physical Computing, Embedded AI, Robotics & Cognitive IoT.

Tutor: Caique Nonato da Silva Bezerra

**SÃO PAULO
2026**



SUMÁRIO

1. INTRODUÇÃO	1
2. DESENVOLVIMENTO 1 – WOKWI E ESP32.....	2
2.1 Objetivo da segunda etapa	2
2.2. Contextualização da primeira etapa.	2
2.3 Funcionamento Wi-Fi e o MQTT	3
3. DESENVOLVIMENTO 2 – NODE-RED	6
4. DESENVOLVIMENTO 3 – INFLUXDB CLOUD	9
CONCLUSÃO.....	11
FONTES	12



FIGURAS

FIGURA 01 – Primeira etapa	3
FIGURA 02 – Condições de risco	4
FIGURA 03 – Fluxo Node-RED.....	6
FIGURA 04 – Tela do Dashboard feito pelo nó Node-RED Dashboard	8
FIGURA 05 – Tela Bucket criado	9
FIGURA 06 – Tela Data Explorer	10



CÓDIGO

CÓDIGO 01 – Configuração Wi-Fi SIDD e PASSWORD	3
CÓDIGO 02 – Estrutura formato JSON	5



IoT - Internet of Things (Internet das Coisas)

RAM - Random Access Memory (Memória de Acesso Aleatório)

MQTT - Message Queuing Telemetry Transport

BPM - Batimentos por minuto

GPIO - General Purpose Input/Output (Entrada/Saída de Propósito Geral)

JSON - JavaScript Object Notation (Notação de Objeto JavaScript)

TLS - Transport Layer Security (Segurança da Camada de Transporte)



RESUMO

Este relatório apresenta o desenvolvimento da segunda etapa do projeto Cardiola, voltada à transmissão de dados simulados de saúde para a nuvem e visualização em tempo real. O sistema utiliza um ESP32 simulado no ambiente Wokwi, responsável pela leitura de temperatura, umidade, batimentos cardíacos simulados por potenciômetro, detecção de movimento e botão de emergência. Os dados são estruturados em formato JSON e enviados via protocolo MQTT para o broker HiveMQ Cloud. Em seguida, o Node-RED atua como cliente MQTT, recebendo as mensagens, convertendo o *payload* e exibindo as informações em uma dashboard com gráfico, gauge e indicadores textuais de alerta. Como expansão do sistema, os dados também foram armazenados no InfluxDB Cloud, permitindo histórico das medições e futura integração com ferramentas analíticas avançadas. O projeto demonstra a aplicação prática de conceitos de Internet das Coisas, *Fog Computing*, *Cloud Computing*, comunicação MQTT e monitoramento remoto em saúde digital.

Palavras-chave: ESP32. MQTT. Node-RED. InfluxDB. Saúde Digital. IoT.



ABSTRACT

This report presents the development of the second stage of the Cardiola project, focused on transmitting simulated health data to the cloud and visualizing it in real time. The system uses an ESP32 simulated in the Wokwi environment, responsible for reading temperature, humidity, heart rate simulated by a potentiometer, movement detection, and emergency button status. The data is structured in JSON format and sent via the MQTT protocol to the HiveMQ Cloud broker. Then, Node-RED acts as an MQTT client, receiving the messages, converting the payload, and displaying the information on a dashboard with a chart, gauge, and textual alert indicators. As an extension of the system, the data was also stored in InfluxDB Cloud, enabling historical records and future integration with advanced analytical tools. The project demonstrates the practical application of Internet of Things, Fog Computing, Cloud Computing, MQTT communication, and remote monitoring concepts in digital health.

Keywords: ESP32. MQTT. Node-RED. InfluxDB. Digital Health. IoT.



1. INTRODUÇÃO

A internet das Coisas, também conhecida como *IoT*, possibilita a conexão de dispositivos físicos à internet, permitindo a coleta, transmissão, processamento e visualização de dados em tempo real. No contexto da saúde digital, essa tecnologia pode ser aplicada em sistemas de monitoramento remoto, como pulseiras inteligentes, sensores corporais e dispositivos hospitalares conectados.

A atividade desenvolvida teve como objetivo de coleta de dados simulados. Os dados coletados foram enviados para a nuvem por meio do protocolo MQTT, utilizando o HiveMQ Cloud como broker. Em seguida, as informações foram recebidas pelo Node-RED, onde foram tratadas e exibidas em um dashboard em tempo real.

Além de visualização local no Node-RED, o projeto também foi integrado ao InfluxDB Cloud, banco de dados especializado no armazenamento de séries temporais. Dessa forma, o projeto passou a contemplar não apenas o acompanhamento instantâneo dos sinais vitais, mas também o armazenamento histórico dos dados para futuras análises.

A arquitetura desenvolvida representa uma aplicação prática dos conceitos de *Fog Computing* e *Cloud Computing*. A camada de *Fog Computing* é representada pelo processamento local realizado no ESP32, que identifica condições de alerta antes do envio dos dados. A camada de *Cloud Computing* é representada pelo uso do HiveMQ Cloud e do InfluxDB. A camada de *Cloud Computing* é representada pelo uso do HiveMQ Cloud e do InfluxDB Cloud, responsáveis pela comunicação e armazenamento dos dados na nuvem.



2. DESENVOLVIMENTO 1 – WOKWI E ESP32

2.1 Objetivo da segunda etapa

O primeiro ambiente utilizado no projeto foi o Wokwi, uma plataforma de simulação de circuitos eletrônicos que permite executar projetos com microcontroladores, sensores e atuadores sem a necessidade imediata de componentes físicos. No projeto CardiolA, o Wokwi foi utilizado para simular o funcionamento do ESP32 e dos componentes responsáveis pela geração dos dados de saúde.

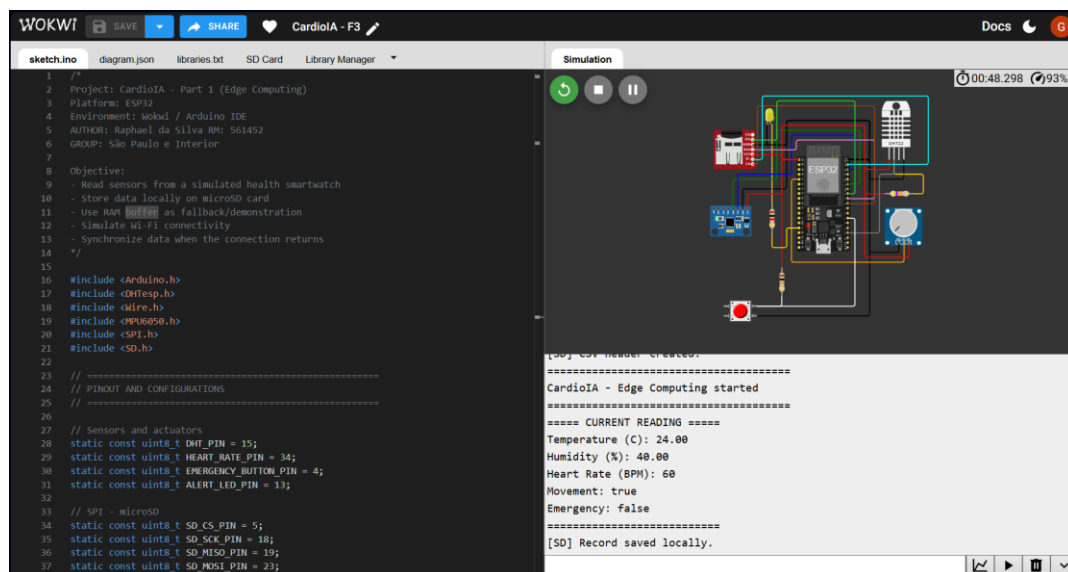
O ESP32 foi utilizado como elemento central do sistema. Ele ficou responsável pela leitura dos sensores simulados, processamento local dos dados, identificação de situação de alerta e envio de informações para nuvem por meio do protocolo MQTT.

2.2. Contextualização da primeira etapa.

Na etapa inicial, o sistema priorizou a computação em borda, utilizando o ESP32 para realizar a coleta, organização e avaliação local dos dados. O protótipo contou com o sensor DHT22 para leitura de temperatura e umidade, o MPU6050 para detecção de movimento, um potenciômetro para simulação dos batimentos cardíacos e um botão de emergência para representar eventos críticos manuais. Além disso, foram implementados mecanismos de armazenamento local, como cartão microSD e buffer em memória *RAM*, permitindo que os dados fossem preservados temporariamente em situações de indisponibilidade de conexão.



FIGURA 01 – Primeira etapa



Fonte: Própria Autoria

2.3 Funcionamento Wi-Fi e o MQTT

O Código desenvolvido para o ESP32 realiza a conexão com a rede Wi-Fi simulada no Wokwi utilizando a rede:

CÓDIGO 01 – Configuração Wi-Fi SIDD e PASSWORD

```
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";
```

Fonte: Própria Autoria

Após a conexão Wi-Fi, o ESP32 se conecta ao broken MQTT do HiveMQ Cloud por meio de uma conexão segura na porta 8883. Para isso, foram utilizadas bibliotecas *WiFi.h*, *WiFiClientSecure.h* e *PubSubClient.h*.

A leitura dos sensores é feita periodicamente a cada dois segundos. Durante cada ciclo de leitura, o sistema coleta:

- Temperatura;
- Umidade;
- Batimento cardíaco;
- Movimento;



- Estado botão de emergência.

Após coleta dos dados, O ESP32 avalia se existe alguma condição de risco.

As regras utilizadas foram:

FIGURA 02 – Condições de risco

	TEMPERATURA	>	38°C
	BATIMENTO CARDÍACO	>	120 bpm
	BOTÃO DE EMERGÊNCIA	>	True (Verdadeiro)

Fonte: Própria Aatoria

A estrutura dos dados foi organizada em um registro contendo tempo de exceção, temperatura, umidade, batimento cardíaco, movimento, emergência e alerta. O código define a estrutura *HealthRecord*, responsável por armazenar as informações principais antes de conversão para JSON e envio via MQTT.

A leitura dos batimentos cardíacos foi simulada por meio de um potenciômetro conectado ao GPIO 34. Como o GPIO 34 é entrada analógica do ESP32 Devkit V1, o valor lido foi convertido para uma faixa aproximada de 60 a 140 batimentos por minutos, simulando uma variação de frequência cardíaca.

A lógica de alerta foi baseada em limites definidos no sistema. A temperatura acima de 38°C, batimentos cardíacos acima de 120 BPM ou acionamento do botão de emergência ativam o estado de alerta. Essa análise local representa a aplicação do conceito de *Fog Computing*, pois o próprio dispositivo realiza uma primeira avaliação dos dados antes de transmiti-los à nuvem.

A mensagem enviada pelo ESP32 foi estruturada em formato JSON.



CÓDIGO 02 – Estrutura formato JSON

```
{  
  "timestamp_ms": 38109,  
  "temperature_c": 24.00,  
  "humidity_percent": 40.00,  
  "heart_rate_bpm": 60,  
  "movement_detected": true,  
  "emergency_pressed": false,  
  "alert_active": false  
}
```

Fonte Própria Autoria

O envio das mensagens foi realizado por meio da função de publicação *MQTT*, que transforma o registro de saúde em JSON e publica o *payload* no tópico configurado. A função de criação do JSON inclui os campos de temperatura, umidade frequência cardíaca, movimento, emergência e alerta.

Durante os testes no Wokwi, o Monitor Serial foi utilizado para validar a conexão Wi-Fi, a conexão MQTT e a publicação dos dados. O sistema apresentou mensagem de confirmação indicando que os dados foram publicados com sucesso no tópico *cardioia/health/data*.

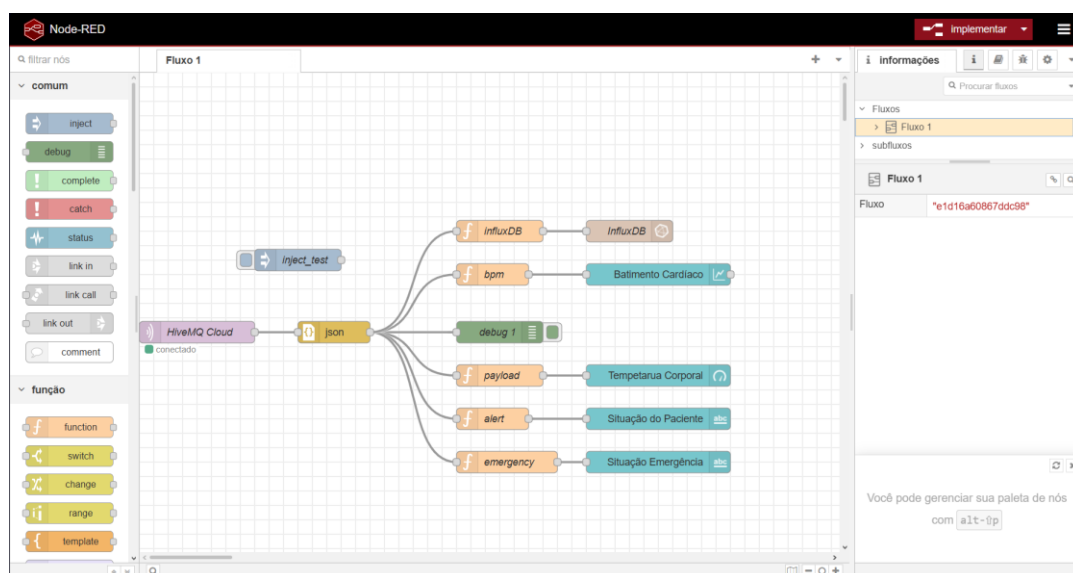


3. DESENVOLVIMENTO 2 – NODE-RED

Após a configuração do ESP32, foi implementada a etapa de recepção, tratamento e visualização dos dados no Node-RED. O Node-RED foi utilizado como cliente MQTT, assinando o tópico publicado pelo ESP32 e encaminhando os dados recebidos para os elementos visuais de dashboard.

O fluxo principal no Node-RED foi organizado da seguinte forma:

FIGURA 03 – Fluxo Node-RED



Fonte: Própria Autoria

O nó *mqtt in* foi configurado com o *broken* HiveMQ Cloud, utilizando TLS na porta 8883 *cardioia/health/data*. Essa escolha foi importante para enviar o recebimento de mensagens que não fossem JSON, como mensagem de status.

O nó *json* foi utilizado para converter a mensagem recebida em texto para um objeto JavaScript. Com isso, tornou-se possível acessar individualmente os campos do *payload*, como:

`msg.payload.temperature_c`

`msg.payload.heart_rate_bpm`

`msg.payload.alert_active`



Em seguida, foram criadas funções específicas para encaminhar os valores aos componentes de dashboard. Para o medidor de temperatura, foi extraído o campo *temperature_c*. Para o gráfico de batimentos cardíacos, foi extraído o campo *hearth_rate_bpm*. Para os indicadores textuais, foram analisados os campos *alert_active* e *emergency_presed*.

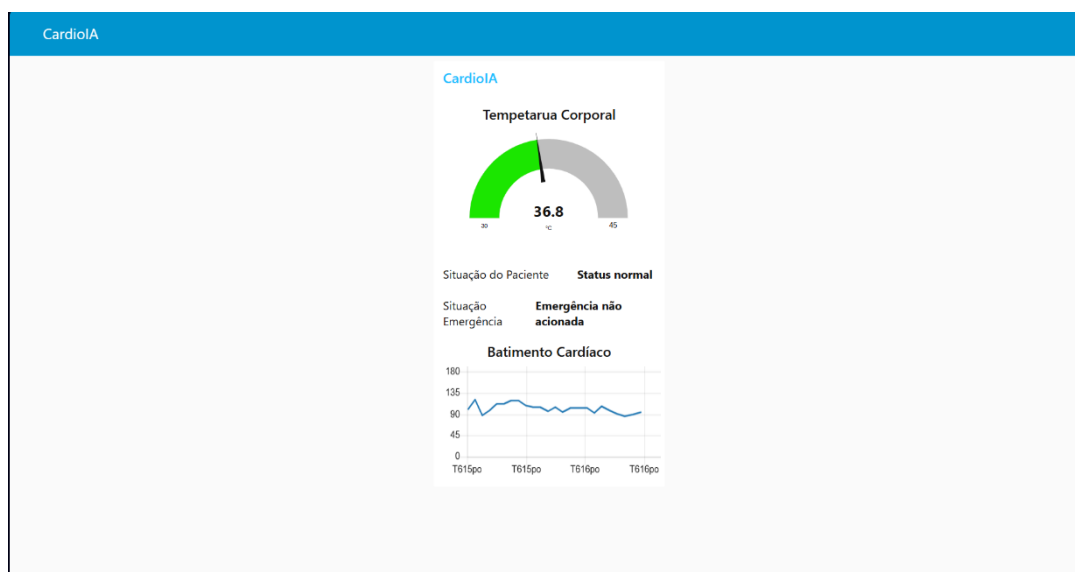
- a) O dashboard no Node-RED foi composta por:
- b) Um medidor do tipo gauge para temperatura corporal;
- c) Um gráfico e linha para batimentos cardíacos;
- d) Um indicador textual da situação do paciente;
- e) Um indicador textual da situação de emergência.

O medidor de temperatura foi configurado com faixa entre 30°C a 45°C, permitindo visualizar variações simuladas pelo sensor DHT22. O gráfico de batimentos cardíacos foi configurado para exibir a variação do BPM em tempo real, permitindo observar a alteração conforme o potenciômetro era movimento no Wokwi.

O indicador de situação do paciente foi configurado para apresentar uma mensagem de condição normal quando os valores permaneciam dentro dos limites definidos e uma mensagem de atenção quando temperatura ou BPM ultrapassavam os limites. O indicador de emergência apresentava estado normal ou emergência acionada, de acordo com o botão físico simulado.



FIGURA 04 – Tela do Dashboard feito pelo nó Node-RED Dashboard



Fonte: Própria Autoria

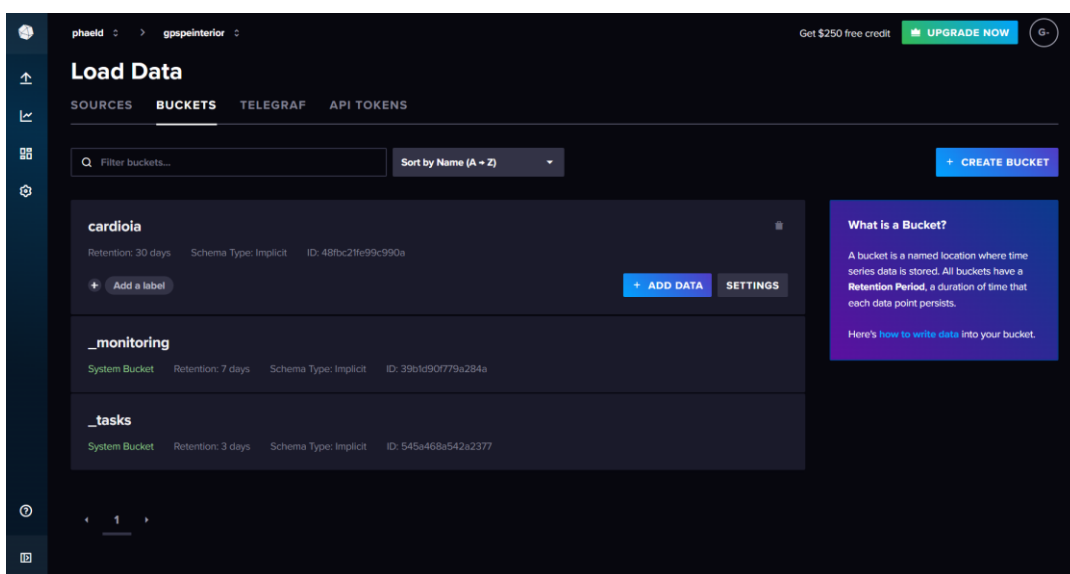


4. DESENVOLVIMENTO 3 – INFLUXDB CLOUD

Além da visualização em tempo real no Node-RED, foi realizada a integração com o InfluxDB Cloud para armazenamento histórico dos dados. Essa etapa ampliou a aplicação, permitindo que os dados transmitidos pelo ESP32 fossem persistidos em um banco de dados de séries temporais.

O InfluxDB Cloud foi configurado com *bucket* denominado cardioia. Nesse *bucket* foi criada a *measurement sinais_vitais*. A integração entre Node-RED e InfluxDB foi realizado por meio de um nó de saída do InfluxDB, conectado ao fluxo após o tratamento do JSON.

FIGURA 05 – Tela Bucket criado



Fonte: Própria Autoria

A função de preparação dos dados foi responsável por transformar o *payload* recebido em um formato compatível com o InfluxDB. Os valores dos sensores foram enviados como Fields, enquanto identificadores do projeto foram enviados como tags. Com essa estrutura, os dados de temperatura, umidade, BPM, movimento, emergência e alerta passaram a ser gravados como campos consultáveis no InfluxDB.



FIGURA 06 – Tela Data Explorer

phaeld > gpspainterior

Get \$250 free credit [UPGRADE NOW](#)

Data Explorer

+ New Script OPEN SAVE

Schema Browser SQL Sync Bucket cardiola Measurement sinais_vitals

Search fields and tag keys

- Fields
 - alert_active
 - emergency_pressed
 - heart_rate_bpm
 - humidity_percent
 - ☒ measurement
 - movement_detected
 - temperature_c
- Tag Keys
 - device
 - project

```
1 SELECT *
2 FROM "sinais_vitals"
3 WHERE
4 time >= now() - interval '24 hour'
```

Ready (253ms) CSV Past 2d RUN

Search results... 12 rows TABLE GRAPH

alert_active	device	emergency_pressed	heart_rate_bpm	humidity_percent	measurement	movement_detected
no group double	no group string	no group double	no group double	no group double	no group string	no group double
0	esp32_cardioia	0	107	40		1
0	esp32_cardioia	0	107	40		1
0	esp32_cardioia	0	91	40		1
0	esp32_cardioia	0	91	40		1
1	esp32_cardioia	1	91	40		1

1 2

Fonte: Própria Autoria



CONCLUSÃO

O desenvolvimento da segunda etapa do projeto CardiolA permitiu implementar um sistema completo de monitoramento simulado de sinais vitais, utilizando ESP32, MQTT, HiveMQ Cloud, Node-RED e InfluxDB Cloud. O sistema atendeu aos requisitos propostos ao realizar a transmissão de dados simulados do ESP32 para a nuvem, a recepção via protocolo MQTT e a visualização em tempo real por meio de uma dashboard.

No ambiente Wokwi, o ESP32 foi responsável pela coleta dos dados simulados de temperatura, umidade, batimentos cardíacos, movimento e emergência. A lógica local de alerta permitiu identificar condições de risco antes mesmo do envio dos dados, caracterizando a aplicação de *Fog Computing*.

No Node-RED, os dados foram recebidos via MQTT, convertidos de JSON para objeto e direcionados para elementos visuais, como gráfico de batimentos cardíacos, medidor de temperatura corporal e indicadores de alerta. Essa etapa validou a comunicação em tempo real entre o dispositivo IoT e o sistema de visualização.

A integração com o InfluxDB Cloud permitiu armazenar os dados em um banco de séries temporais, ampliando o projeto para além da visualização instantânea. Com isso, tornou-se possível registrar o histórico dos sinais vitais e preparar a aplicação para futuras análises em ferramentas avançadas.

Conclui-se que o projeto atingiu o objetivo proposto e demonstrou, de forma prática, conceitos fundamentais de Internet das Coisas, MQTT, *Fog Computing*, *Cloud Computing* e saúde digital. A solução desenvolvida representa um protótipo funcional de monitoramento remoto, podendo ser expandida futuramente com autenticação, dashboards analíticos, notificações automatizadas e modelos de inteligência artificial para detecção de riscos.



FONTES

Faculdade de Informática e Administração Paulista (FIAP). Código Organizado, Projeto Seguro: POO Na Prática Com Arduino e ESP32. São Paulo: 2025.

Faculdade de Informática e Administração Paulista (FIAP). IoT Cognitivo: Que Falam e Pensam. São Paulo: 2026.

Faculdade de Informática e Administração Paulista (FIAP). Cloud Computing como pilar da IoT moderna. São Paulo: 2026.

Faculdade de Informática e Administração Paulista (FIAP). Inteligência na Borda: Fog e Edge Computing Transformando a IoT. São Paulo: 2026.

HIVEMQ. MQTT Essentials.

Disponível em: <https://www.hivemq.com/mqtt>. Acesso em: 01/05/2026.

NODE-RED. Node-RED Documentation.

Disponível em: <https://nodered.org/docs/>. Acesso em: 01/05/2026

INFLUXDATA. InfluxDB Cloud Documentation.

Disponível em: <https://docs.influxdata.com>. Acesso em: 02/05/2026.

